

TP : Introduction à la Sobriété Numérique & Performance

Cas d'étude : Simulateur de Galaxies (N-Body)

1. Contexte et Objectifs

Dans un monde où le numérique représente une part grandissante de la consommation électrique mondiale, l'ingénieur de demain ne doit plus seulement chercher à ce que son code "fonctionne". Il doit chercher à ce qu'il soit **performant** et **économe en énergie**.

Ce TP a pour but de vous initier :

- Aux problématiques de performance et d'efficacité énergétique en calcul scientifique.
- À la comparaison de plusieurs langages (C++, Python, Octave) pour un même algorithme.
- À l'influence de l'architecture matérielle (PC vs Jetson Orin Nano).

Le support expérimental est un simulateur de galaxies basé sur le calcul des interactions gravitationnelles entre particules. Le simulateur repose sur un algorithme naïf où chaque particule interagit avec toutes les autres, avec une complexité en $O(N^2)$.

2. Environnement de Travail

Vous allez travailler sur deux plateformes distinctes pour comparer les résultats :

1. **PC de TP** : Puissant mais potentiellement énergivore.
2. **Nvidia Jetson Orin Nano (ARM)** : Architecture embarquée basse consommation.

Le projet vous est fourni avec une structure standardisée pour chaque langage :

- `src/` : Codes sources (`Main.cpp`, `Galaxy.hpp`, `RenderNaive.hpp`, etc.).
- `Makefile` : Outil central pour compiler, exécuter, analyser et nettoyer le projet.

3. Travail Préparatoire : Compréhension

Avant de lancer les mesures, inspectez le code. Comprenez l'algorithme et d'où vient la complexité calculatoire.

1. Regardez le fichier `RenderNaive.hpp`. Il contient l'implémentation de l'algorithme naïf de calcul des forces.
2. Notez que les autres langages implémentent exactement la même logique algorithmique.

4. Manipulation : Protocole Expérimental

Pour **chaque langage** (C++, Rust, Java, Julia, Python, Octave) et sur **chaque plateforme** (PC et Jetson), réalisez les étapes suivantes.

Étape A : Compilation et Vérification

Utilisez le `Makefile` fourni. Il est votre outil d'automatisation.

1. Dans le dossier du langage choisi, lancez : `make build` pour compiler le projet et générer les exécutable.
2. Lancez `make diff` pour comparer les sorties avec le langage de référence et vérifier la validité scientifique des résultats numériques.

- o *Si le test échoue, vos mesures d'énergie ne seront pas valides.*

Étape B : Mesure de performance et d'énergie

Nous allons utiliser le mode "mesures" qui lance une exécution longue (environ 120 secondes) pour stabiliser les mesures.

1. Lancez la commande : `make experimental`.
2. Le programme va exécuter les simulations et générer des fichiers de résultats contenant :
 - o Le temps par itération.
 - o La consommation énergétique.

5. Analyse et Compte-rendu

C'est le cœur du TP. Vous devez documenter vos résultats dans un rapport court (Markdown ou PDF) répondant aux questions suivantes, appuyé par des graphiques ou tableaux.

Comparaison des Langages (C++, Rust, Java, Julia, Python, Octave) :

- Quel langage est le plus rapide ? Quel sont les facteurs de ralentissement entre les différents langages ?
- Quel langage consomme le plus d'énergie pour le **même** calcul ?

Comparaison des Plateformes (PC vs Jetson) :

- Le PC calcule-t-il plus vite que la Jetson ?
- Laquelle des deux plateformes est la plus efficace énergétiquement (c'est-à-dire : qui utilise le moins de Joules par itération pour finir le travail) ?

- o *Rappel : Énergie (J) = Puissance (W) × Temps (s) / nb_itérations (n).*

Conclusion :

- Ce projet vous a-t-il aidé à développer votre esprit critique face aux résultats ?
- Si vous deviez déployer ce simulateur sur un système embarqué (batterie limitée), quel couple Langage/Plateforme choisiriez-vous et pourquoi ?